API Resource Modeling & URI Design

nesting, actions and bulk operations

Watch the 4-min video

In one line: a good API is predictable: plural nouns and IDs, one level of nesting for true ownership, query parameters for filters, explicit POST actions for non-CRUD operations, and capped bulk endpoints with per-item results.

1 Naming rules

If a developer can **guess your endpoint before reading the docs**, you have modeled it well. Start with four rules:

- **Plural nouns** for collections, an **ID** for each item: `/orders` and `/orders/1001`.
- **Lowercase with hyphens** in paths: `/shipping-addresses`, not `/ShippingAddresses` or `/shipping_addresses`.
- **One casing style for JSON fields**, `snake_case` or `camelCase`, used everywhere.
- **Version up front**: `/v1/...`.

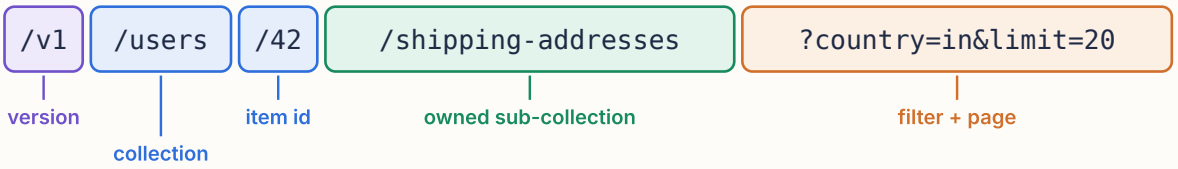


Figure 1. Every part of the URI has one job. Reading it left to right tells you the version, the resource, the owner and the filter.

2 Nested resources: one level, true ownership

Nest only when the child truly belongs to the parent, and keep it to one level. `/users/42/addresses` makes sense: an address belongs to one user, and the path **scopes authorization naturally** (check that the caller is user 42, then everything below is theirs).

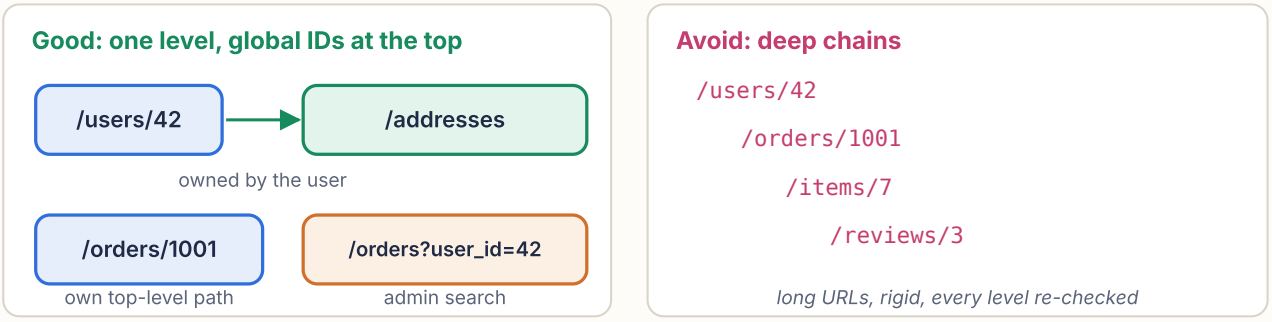


Figure 2. Left: nest owned children one level deep and promote globally identified resources to the top. Right: a chain that forces clients to know four IDs to reach one review.

Avoid deep chains such as `/users/42/orders/1001/items/7/reviews/3`. Once a resource has its own **global ID**, give it a **top-level path**: `/orders/1001`. Order 1001 is unique on its own, so the user ID in the

path adds nothing but length and another check.

3 Sub-resource or query parameter?

Use	Example	Meaning
Sub-resource	`/users/42/orders`	Ownership: the orders that belong to user 42
Query parameters	`/orders?status=shipped&created_after=2026-10-01`	Filter a collection by attributes
Both	`/orders?user_id=42`	Handy for admin tools that search across users

Rule of thumb: if removing the parent should make the child meaningless, it is a sub-resource. If it is just an attribute you search by, it is a query parameter.

4 Non-CRUD actions: cancel, refund, resend

Do not force actions into a fake field update. Two good options:

- **Custom method**: `POST /v1/orders/1001:cancel`, the style used in Google's APIs.
- **Action as a resource**: `POST /v1/orders/1001/cancellations`. It also **keeps a record** of who cancelled and why, which you can later `GET`.

Either way it is a **POST**, it must be **idempotent** (a retried cancel does not refund twice), and it returns **409 Conflict** if the order can no longer be cancelled.

Why not just PATCH the status?

Because a cancel is not a simple field change. It **triggers refunds, restocks inventory and sends emails**. A PATCH that sets `status` to `cancelled` hides all that behavior, makes validation awkward, and invites clients to set any status they like, such as `shipped`. Explicit actions make **intent and permissions clear**: "may cancel" is a different permission from "may edit the delivery note".

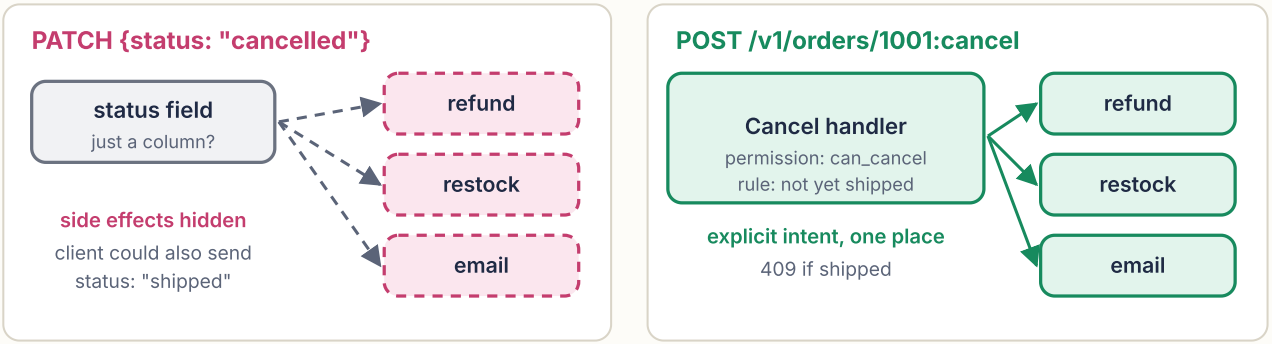


Figure 3. The same business operation. On the left the side effects hang off a field write; on the right one handler owns the rules, the permission and the effects.

5 Bulk operations

Creating a thousand items one call at a time is slow: 1,000 calls × 50 ms is 50 seconds of round trips. Offer a **bulk endpoint**, such as `POST /v1/orders:batchCreate`, with a **cap on items per request** (say 500).

- **Return a result for each item**, because some may succeed while others fail.

- For **very large jobs**, accept with **202 Accepted** and let clients **poll a job resource** for progress.

WORKED EXAMPLE: PARTIAL FAILURE

A client sends 3 orders in one batch. The second has an unknown SKU. The response reports each item, in request order:

```
POST /v1/orders:batchCreate        → 200 OK
{ "results": [
  { "index": 0, "status": 201, "id": "1001" },
  { "index": 1, "status": 422, "error": "unknown sku ZX-9" },
  { "index": 2, "status": 201, "id": "1002" } ] }
```

The client retries only index 1 after fixing the SKU. A batch of 2,000 items exceeds the cap of 500, so it gets `400` with a clear message, and the client splits it into 4 requests.

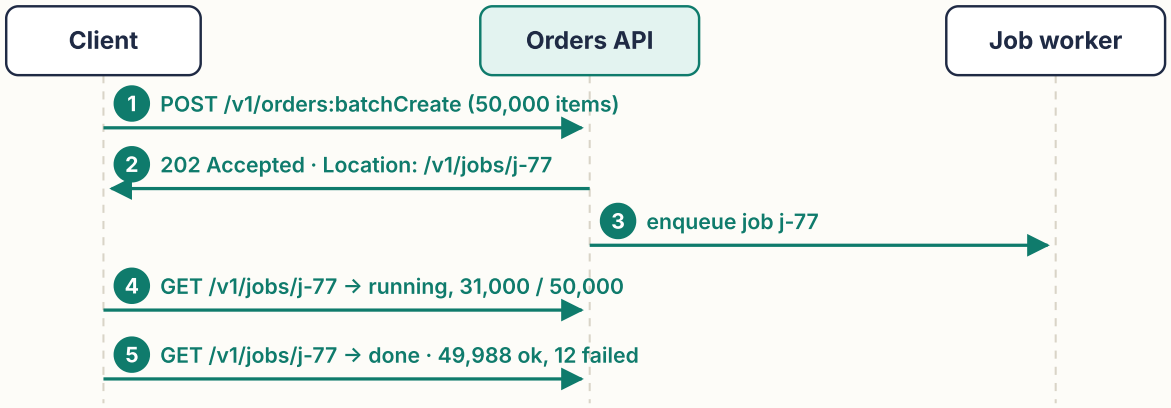


Figure 4. Large jobs run asynchronously. The 202 response points to a job resource the client polls; the final job state lists the failed items.

COMMON MISTAKES

- Mixing styles: `/Orders`, `/order_items` and `/shippingAddresses` in one API.
- Nesting three or four levels deep, so clients must know every parent ID.
- Modeling cancel as `PATCH status`, then validating every possible status transition by hand.
- A bulk endpoint that returns one overall "failed" when 1 of 500 items was bad.
- Unbounded batches: one 1 GB request ties up a server and its database.

6 Key terms

Collection: plural noun, e.g. /orders

Sub-resource: owned child, e.g. /users/42/addresses

Global ID: unique on its own; earns a top-level path

Query parameter: filter, sort or page a collection

Custom method: POST /orders/1001:cancel

Action resource: POST /orders/1001/cancellations

Bulk endpoint: many items, one call, per-item results

Job resource: tracks a long-running request

202 Accepted: accepted, will finish later

7 Interview questions

Q1. How do you design endpoints for nested resources? BASIC

Nest one level only for true ownership, like /users/42/addresses. Give resources with global IDs their own top-level path (/orders/1001), and use query parameters for filtering (/orders?user_id=42&status=shipped).

Q2. What naming rules make an API predictable? BASIC

Plural nouns plus IDs (/orders/1001), lowercase hyphenated paths (/shipping-addresses), one JSON casing everywhere, and a version prefix (/v1).

Q3. How do you expose non-CRUD actions like cancel or refund? SCENARIO

As a POST to a custom method (/v1/orders/1001:cancel) or to an action resource (/v1/orders/1001/cancellations, which also records who and why). Make it idempotent and return 409 if the order can't be cancelled.

Q4. A customer must import 10,000 products. Design the API. SCENARIO

POST /v1/products:batchCreate capped at 500 per request with per-item results (20 calls), or one async job: 202 Accepted with Location /v1/jobs/{id}, polled for progress and a final list of failures.

Q5. Why not just PATCH the order status to cancelled? TRICKY

Cancel triggers refunds, restocking and emails; a field update hides that, makes validation awkward, and lets clients set any status, like shipped. An explicit action makes intent, rules and permissions clear.

Q6. Sub-resource or query parameter: /users/42/orders or /orders?user_id=42? TRICKY

Both can exist. The sub-resource expresses ownership and scopes authorization to user 42; the query parameter suits admin tools searching across users. Expose the second only to callers allowed to see other users' orders.

Next: SD-C-006 · API Versioning Without Breaking Clients backendroadmap.com